PROJECT REPORT

AutoHealOps

Automated DevOps Monitoring and Self-Healing Demonstration

Submitted by

Sindhupriya Abbineni

Computer Science and Engineering

Repository	github.com/sindhupriya5e1/AutoHealOps
Live dashboard	https://autohealops-demo.onrender.com/
Monitoring	Prometheus + Grafana public demo services

A practical portfolio project demonstrating metrics collection, observability, optional alerting and local container recovery.

Abstract

AutoHealOps is a containerized DevOps monitoring and recovery demonstration. A Python FastAPI application collects CPU, memory and disk utilization using psutil and exposes the values in Prometheus format. Prometheus scrapes the public FastAPI metrics endpoint, while Grafana uses Prometheus as its data source to visualize resource usage. Optional Telegram notifications can be sent when configured thresholds are crossed. A PowerShell watchdog runs on the local host, checks the local application health endpoint and attempts to restore the Docker Compose application when it becomes unavailable.

1. Introduction

Modern applications need continuous visibility into resource consumption and service health. When an application becomes unavailable or resource usage becomes unusually high, developers need a way to detect the condition and take corrective action. AutoHealOps brings together a lightweight monitoring stack and a controlled recovery workflow to demonstrate these DevOps practices.

2. Problem Statement

Without monitoring, a developer may not notice CPU, memory or disk pressure and may discover an application failure only after a user reports it. Manual recovery can also take time. This project demonstrates a repeatable workflow for collecting metrics, visualizing them, notifying an operator and attempting to restore a failed local application container.

3. Objectives

- Measure CPU, memory and disk utilization.
- Expose metrics in Prometheus format and scrape them using Prometheus.
- Visualize resource usage in Grafana dashboards.
- Support optional Telegram threshold notifications.
- Check local application health using a PowerShell watchdog.
- Demonstrate recovery of a stopped or unhealthy local Docker Compose service.
- Publish public URLs for reviewers to inspect the API and monitoring services.

4. Project Scope

The public deployment provides a FastAPI dashboard/API, a Prometheus scraper and a Grafana dashboard. Public metrics describe the Render-hosted FastAPI container. The PowerShell watchdog and Docker Compose recovery workflow run on the local project host. Public cloud recovery of the Render service is not implemented in this version.

5. System Architecture

Public monitoring flow: Browser → FastAPI dashboard/API → /metrics → Prometheus → Grafana dashboard.

Local recovery flow: Docker Compose app → /health check → PowerShell watchdog → docker compose up -d --no-deps app.

- FastAPI serves the dashboard, health endpoint, project metadata and metrics.
- psutil reads CPU, memory and disk utilization.
- The application updates Prometheus gauges and checks alert thresholds.
- Prometheus periodically scrapes the FastAPI /metrics endpoint.
- Grafana queries Prometheus and renders resource monitoring panels.
- Telegram sends threshold notifications only when bot credentials are configured.

- The local PowerShell watchdog checks health every 30 seconds and attempts Docker Compose recovery if the check fails.

6. Technology Stack

Technology	Purpose
Python 3.12	Application runtime
FastAPI + Uvicorn	REST API and dashboard
psutil	CPU, memory and disk measurements
prometheus-client	Prometheus metric exposition
Prometheus	Metrics scraping and querying
Grafana	Monitoring dashboards
Docker + Docker Compose	Containerization and local orchestration
PowerShell	Local host watchdog and recovery
Telegram Bot API	Optional notifications
GitHub + Render	Source control and public demo hosting

7. Implementation Details

7.1 FastAPI endpoints

Endpoint	Purpose
/	Main dashboard
/health	Health status JSON
/api/info	Project metadata and deployment scope
/api/metrics	CPU, memory and disk values as JSON
/metrics	Prometheus exposition format
/api/monitoring	Prometheus service and scrape-target status
/docs	Interactive API documentation

7.2 Metrics and alert thresholds

The application exposes system_cpu_percent, system_memory_percent and system_disk_percent.

Metric	Threshold
CPU utilization	80%
Memory utilization	80%
Disk utilization	85%

Telegram notifications are optional. The application skips sending messages if credentials are absent.

7.3 Prometheus and Grafana

The public Prometheus configuration scrapes the hosted FastAPI /metrics endpoint. Grafana is provisioned with a Prometheus data source and an AutoHealOps dashboard containing CPU, memory and disk gauges and a resource-usage time-series panel. The local Prometheus configuration scrapes the app service on the Docker Compose network.

7.4 Local self-healing watchdog

The PowerShell watchdog polls the local /health endpoint every 30 seconds. If the health request fails, it runs docker compose up -d --no-deps app to bring the app service back. It can send failure and recovery notifications when Telegram credentials are configured.

> This is a local-host recovery demonstration. It does not restart the public Render service.

8. Public Deployment

Service	Public URL
Main dashboard	https://autohealops-demo.onrender.com/
Prometheus UI	https://autohealops-prometheus-live.onrender.com/
Grafana dashboard	https://autohealops-grafana-live.onrender.com/d/autohealops-live/autohealops-live?or

The main page includes links to health, metrics, API documentation, Prometheus and Grafana. Prometheus scrapes the public FastAPI metrics endpoint and Grafana is configured with the Prometheus data source and AutoHealOps dashboard.

9. Local Setup and Demonstration

Prerequisites: Docker Desktop or Docker Engine, Docker Compose and PowerShell.

Start the local services:

```
docker compose up -d --build
docker compose ps
```

Run the watchdog in a separate PowerShell window from the repository root:

```
.\autoheal.ps1
```

Simulate an application failure from another terminal:

```
docker compose stop app
```

After the watchdog's next check, verify recovery:

```
docker compose ps
Invoke-RestMethod http://localhost:8000/health
```

The watchdog checks every 30 seconds, so recovery is not necessarily immediate.

10. Verification and Test Cases

Test	Expected behavior
Open main dashboard	Dashboard loads and requests hosted metrics.
GET /health	Returns healthy status JSON when app is running.
GET /metrics	Returns Prometheus-formatted metrics.
Prometheus scrape	Prometheus periodically requests the public /metrics endpoint.

Test	Expected behavior
Open Grafana	Grafana displays CPU, memory and disk panels when metrics are available.
Stop local app container	Watchdog detects failed health check and requests Docker Compose recovery.
Telegram alert	Messages are sent only when valid bot token and chat ID are configured.

Deployment logs confirmed successful FastAPI responses for the root, health and metrics endpoints. Render logs also confirmed Prometheus and Grafana services started and Grafana provisioned its Prometheus data source and dashboard. Review the Prometheus Targets page to confirm the current scrape target state during a live demonstration.

11. Limitations

- Public metrics describe the Render-hosted API container, not the developer's laptop.
- The PowerShell watchdog and Docker recovery workflow are local-only; cloud self-healing is not included.
- Telegram notifications require a bot token and chat ID.
- Render free services may sleep after inactivity.
- Metrics data on ephemeral storage may be lost after service restarts; persistent retention is not configured.
- This is a portfolio demonstration, not a production monitoring platform.

12. Conclusion

AutoHealOps demonstrates practical DevOps concepts including containerization, resource monitoring, metrics exposition, Prometheus scraping, Grafana visualization, optional threshold alerting, health checks and local automated recovery. The public services allow reviewers to explore the dashboard and monitoring tools, while the local environment demonstrates the self-healing workflow.

13. Project Links

- GitHub source repository
- Main live dashboard
- Prometheus UI
- Grafana dashboard
- Actual project evidence PDF
- Architecture diagram PDF